Module - 8:
API, Integration & Security Testing

- Amjad Hossain
25th April, 2026

API Testing Strategies

1. Functional Testing

- Validate endpoints: request → response
- Status codes, headers, payload

API Testing Strategies

1. Functional Testing

- Validate endpoints: request → response
- Status codes, headers, payload

2. Contract Testing (Critical for microservices)

- Prevent breaking consumers
- Example: using Pact

API Testing Strategies

1. Functional Testing

- Validate endpoints: request → response
- Status codes, headers, payload

2. Contract Testing (Critical for microservices)

- Prevent breaking consumers
- Example: using Pact

3. Schema Validation

- JSON Schema / OpenAPI validation
- Prevent silent breaking changes

API Testing Strategies

1. Functional Testing

- Validate endpoints: request → response
- Status codes, headers, payload

2. Contract Testing (Critical for microservices)

- Prevent breaking consumers
- Example: using Pact

3. Schema Validation

- JSON Schema / OpenAPI validation
- Prevent silent breaking changes

4. Negative Testing

- Invalid payloads
- Missing fields
- Unauthorized access

API Testing Strategies

1. Functional Testing

- Validate endpoints: request → response
- Status codes, headers, payload

2. Contract Testing (Critical for microservices)

- Prevent breaking consumers
- Example: using Pact

3. Schema Validation

- JSON Schema / OpenAPI validation
- Prevent silent breaking changes

4. Negative Testing

- Invalid payloads
- Missing fields
- Unauthorized access

5. Idempotency Testing

- Same request → same result (important for payments)

Automated API Testing Tools and Frameworks

Manual + Automated:	**Code-based:**
● Postman ● Newman	● Rest Assured ● Karate

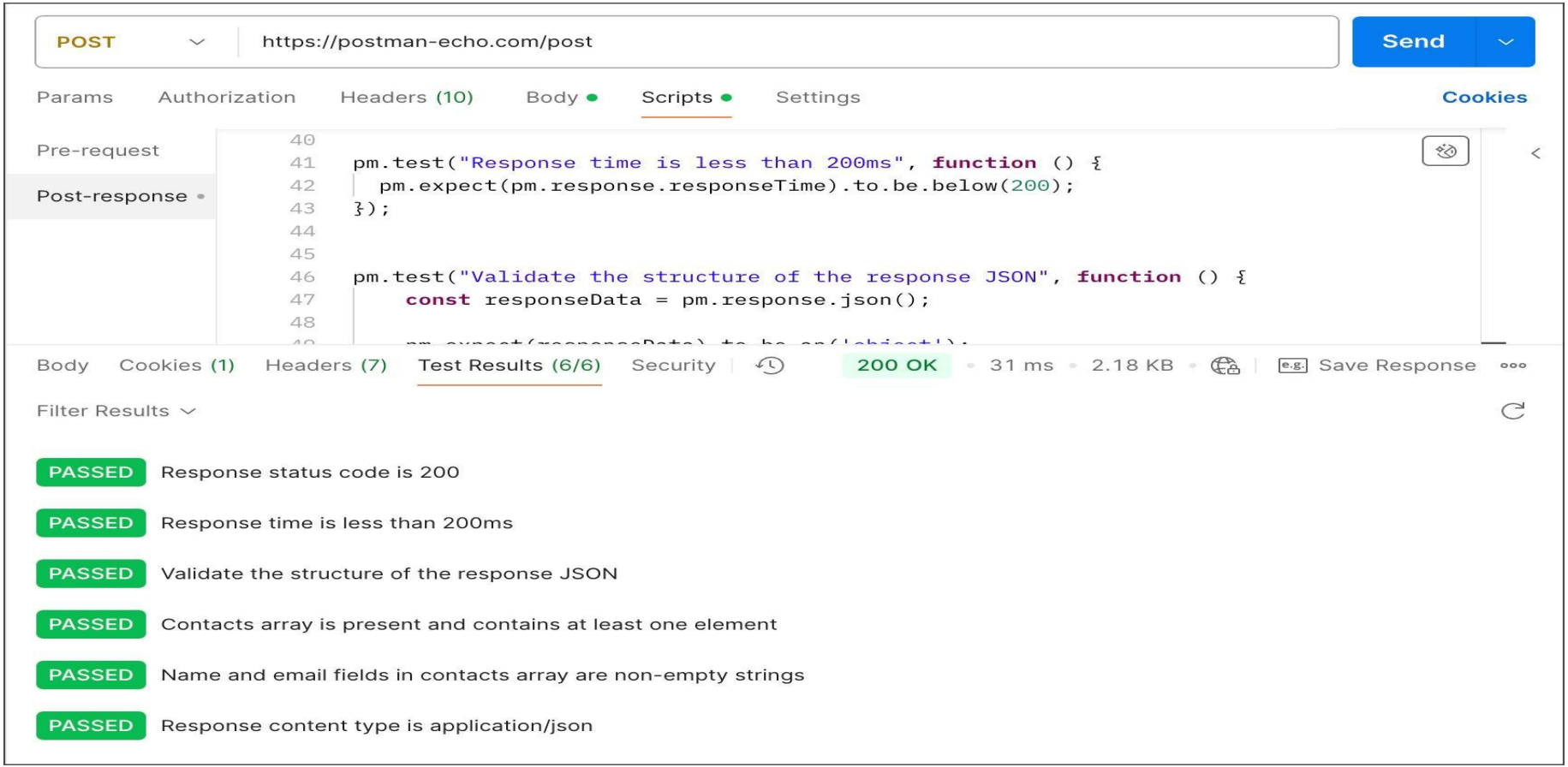
Automated API Testing Tools and Frameworks

Manual + Automated:	**Code-based:**
● Postman ● Newman	● Rest Assured ● Karate

Problem:

1. Backend team pushes a fix by the EOD of Code Freezing day
2. QA finds UI or any feature broken during testing
3. Frontend team start investigation and finds all API(maybe 10) is working fine
4. Frontend team start debugging and after hours, finds that response of a single API has changed
5. Start blaming and fighting
6. Fixes comes later
7. Minimum a single day for QA time is Gone !!

API Testing Strategies



Integration Testing in Distributed Systems

What it is:

Testing **real interaction between services**, not mocks.

Core Problem Space:

Distributed systems introduce failure modes that unit tests never expose:

- **Network unreliability** → latency, packet loss, timeouts
- **Partial failures** → one service down, others alive
- **Data inconsistency** → eventual consistency, replication lag
- **Contract drift** → producer changes response format silently
- **Concurrency issues** → race conditions across services

Types of Integration Testing

1. Service-to-Service Integration

Validate direct communication between services.

Example:

- Order Service → Payment Service → Inventory Service

Focus:

- API contract correctness
- Error handling (timeouts, retries)
- Authentication/authorization propagation

Types of Integration Testing

2. Contract Testing (Consumer-Driven)

Ensures that **API contracts don’t break consumers**.

- Consumer defines expectations
- Provider verifies against those expectations

Types of Integration Testing

2. Contract Testing (Consumer-Driven)

Ensures that **API contracts don't break consumers**.

- Consumer defines expectations
- Provider verifies against those expectations

3. Message-Based Integration Testing

For async systems (event-driven architecture):

Example:

- Service A publishes event → Kafka → Service B consumes

Focus:

- Event schema validation
- Ordering guarantees
- Idempotency

Types of Integration Testing

4. End-to-End (E2E) Integration

Simulates real user workflows across multiple services.

Example:

- User places order → payment → shipping → notification

Focus:

- Entire system correctness
- Real infrastructure behavior

Downside:

- Slow, brittle, hard to debug

Tools:

- Selenium

Performance and Load Testing for Transactions

What it is:

Evaluates how reliably your system handles **real business operations under varying traffic conditions**.

Key Metrics You Must Measure:

1. Throughput (TPS)

- How many transactions per second the system can process
- Critical for scaling decisions

2. Latency (Response Time)

- Average (mean)
- Percentiles (P95, P99 — most important in real systems)

3. Error Rate

- % of failed transactions under load
- Includes timeouts, retries, validation failures

4. Resource Utilization

- CPU
- Memory
- DB connections
- Thread pools

Performance and Load Testing for Transactions

Question:

What tool/s do you use for profiling right now?

Performance and Load Testing for Transactions

Load Generation

- Apache JMeter
- k6
- Gatling

Monitoring & Observability

- Prometheus
- Grafana

Distributed Tracing

- Jaeger

Penetration Testing Fundamentals

What it is:

a **controlled, authorized simulation of real-world attacks** to identify exploitable vulnerabilities in systems, networks, and applications.

Core Objective

Answer these critical questions:

- Can an attacker break in?
- How far can they go?
- What data or systems can be compromised?
- What is the business impact?

Types of Penetration Testing

1. **Black Box Testing**
   - Tester has **no prior knowledge**
   - Simulates external attacker

Types of Penetration Testing

1. **Black Box Testing**
   - Tester has **no prior knowledge**
   - Simulates external attacker

2. **White Box Testing**
   - Full access (code, architecture, credentials)
   - Deep security validation
   - Tests internal logic, flow and structures

Types of Penetration Testing

1. **Black Box Testing**
   - Tester has **no prior knowledge**
   - Simulates external attacker

2. **White Box Testing**
   - Full access (code, architecture, credentials)
   - Deep security validation
   - Tests internal logic, flow and structures

3. **Grey Box Testing**
   - Partial knowledge (most realistic)
   - Common in enterprise environments

Penetration Testing Lifecycle

1. Reconnaissance (Information Gathering) **Goal:** Understand the target ● Domain/IP discovery ● Tech stack identification ● Public exposure **Techniques:** ● Passive (OSINT) ● Active probing	

Penetration Testing Lifecycle

1. Reconnaissance (Information Gathering)	2. Scanning & Enumeration
Goal: Understand the target ● Domain/IP discovery ● Tech stack identification ● Public exposure **Techniques:** ● Passive (OSINT) ● Active probing	**Identify:** ● Open ports ● Services ● Vulnerabilities **Tools:** ● Nmap ● Nikto

Penetration Testing Lifecycle

3. Exploitation Attempt to exploit vulnerabilities **Examples:** ● SQL Injection ● Broken authentication ● Misconfigured access control **Tools:** ● Metasploit ● Burp Suite	

Penetration Testing Lifecycle

3. Exploitation	4. Post-Exploitation
Attempt to exploit vulnerabilities **Examples:** ● SQL Injection ● Broken authentication ● Misconfigured access control **Tools:** ● Metasploit ● Burp Suite	**Assess impact:** ● Privilege escalation ● Data extraction ● Lateral movement

Penetration Testing Lifecycle

3. Exploitation

Attempt to exploit vulnerabilities

Examples:

- SQL Injection
- Broken authentication
- Misconfigured access control

Tools:

- Metasploit
- Burp Suite

4. Post-Exploitation

Assess impact:

- Privilege escalation
- Data extraction
- Lateral movement

5. Reporting

Most critical phase for business:

Includes:

- Vulnerability details
- Exploit steps
- Risk severity
- Remediation guidance

Common Vulnerability Categories

Closely aligned with:

- OWASP Top 10

Examples:

- Injection (SQL, OS)
- Broken Authentication
- Sensitive Data Exposure
- Security Misconfiguration
- Cross-Site Scripting (XSS)

OWASP Top 10

widely accepted, periodically updated list of the most critical security risks to web applications.

It’s used as a baseline for secure development, testing, and auditing.

OWASP Top 10:2025

https://owasp.org/Top10/2025/

A01: Broken Access Control

What it is:

Users can access resources or perform actions outside their permissions.

Demo Scenario:

Perform ID tampering:

```
GET /api/users/123   (valid)
GET /api/users/124   (other user)
```

Expectation:

If other user’s data is returned → vulnerability

A02: Security Misconfiguration

What it is:

Improper configuration of servers, headers, or services exposing the system.

Demo Scenario:

Access hidden/admin endpoints:

```
GET /admin
GET /debug
```

Expectation:

If sensitive endpoints are accessible → vulnerability

A03: Software Supply Chain Failures

What it is:

Compromise via dependencies, packages, or CI/CD pipelines.

Demo Scenario:

Use outdated/vulnerable library or tampered package.

Expectation:

If vulnerable dependency is exploitable → vulnerability

A04: Cryptographic Failures

What it is:

Sensitive data not properly encrypted or protected.

Demo Scenario:

Inspect API response:

- Plain text password
- Weak token
- No HTTPS

Expectation:

If sensitive data is exposed → vulnerability

A05: Injection

What it is

Untrusted input executed as part of a command/query.

Demo Scenario:

```
GET /products?name=' OR '1'='1
```

Expectation:

If all records are returned → vulnerability

A06: Insecure Design

What it is

Business logic flaws due to poor system design.

Demo Scenario

Apply same discount repeatedly:

```
POST /apply-discount
```

Expectation:

If unlimited discount is applied → vulnerability

A07: Authentication Failures

What it is

Weak authentication, session, or credential handling.

Demo Scenario

Perform brute-force login attempts.

Expectation:

If unlimited attempts are allowed → vulnerability

A08: Software & Data Integrity Failures

What it is

Data or code can be modified without validation.

Demo Scenario

```
{ "role": "admin" }
```

Expectation:

If accepted → integrity issue

A09: Logging & Monitoring Failures

What it is

Attacks are not detected, logged, or alerted.

Demo Scenario

Perform multiple failed logins.

Expectation:

If nothing happens → vulnerability

A10: Mishandling of Exceptional Conditions

What it is

System fails improperly under errors or edge cases (fail-open, crashes, leaks).

Demo Scenario

```
GET /api/order?id=null
```

Expectation:

If system crashes or exposes internal data → vulnerability

Server **S**ide **R**equest **F**orgery(often under A01/A10)

What it is

Server makes unintended requests to internal/external systems.

Demo Scenario

{ "url": "http://localhost:8080/admin?**API_KEY=xyz**" }

Expectation:

If internal data returned → SSRF

OWASP Top 10:2025 → Tools Mapping

OWASP Category	Open Source Tool	Commercial Tool
A01: Broken Access Control	OWASP ZAP	Burp Suite
A02: Security Misconfiguration	Nikto	Qualys
A03: Software Supply Chain Failures	OWASP Dependency-Check	Snyk
A04: Cryptographic Failures	testssl.sh	Nessus
A05: Injection	sqlmap	Burp Suite
A06: Insecure Design	OWASP Threat Dragon	Microsoft Threat Modeling Tool
A07: Authentication Failures	Hydra	Burp Suite
A08: Software & Data Integrity Failures	Gitleaks	Checkmarx
A09: Logging & Monitoring Failures	ELK Stack	Splunk
A10: Exceptional Condition Handling	OWASP ZAP	Burp Suite

Open Source Stack (Realistic Setup)

If you want a **fully free stack**, combine:

- OWASP ZAP
- Sqlmap → SQL Injection
- Nikto → infra scanning
- Kali Linux

Kali Linux alone includes hundreds of pentesting tools (600+), covering most attack scenarios

Commercial Stack (Enterprise Reality)

- Burp Suite
- Nessus → infra scanning
- Snyk → DevSecOps
- Splunk → monitoring

These tools provide automation, reporting, and integration needed at scale.

Thank you